Suicide-Risk Detection Using NLP-Based Text and Speech Analysis

A Project Report

Submitted in partial fulfilment of requirements for the award of the degree of
BACHELOR OF TECHNOLOGY
IN
INFORMATION TECHNOLOGY

Submitted by
S. Thanmayi (23761A1251)
Ch. Venu Madhav (24765A1201)
M. Samuel Babu (23761A1235)

Under the Esteemed Guidance of
Dr. G. Rajendra
Associate Professor
Dept. of IT, LBRCE

DEPARTMENT OF INFORMATION TECHNOLOGY
LAKIREDDY BALI REDDY COLLEGE OF ENGINEERING
(AUTONOMOUS)
An ISO 21001:2018, 14001:2015, 50001:2018 Certified Institution
Approved by AICTE, New Delhi and Affiliated to JNTUK, Kakinada
L.B. REDDY NAGAR, MYLAVARAM, NTR DIST., A.P.-521 230
2026-2027

LAKIREDDY BALI REDDY COLLEGE OF ENGINEERING
(AUTONOMOUS)
Accredited by NAAC & NBA (Under Tier - I)
An ISO 21001:2018, 14001:2015, 50001:2018 Certified Institution
Approved by AICTE, New Delhi and Affiliated to JNTUK, Kakinada
L.B. REDDY NAGAR, MYLAVARAM, NTR DIST., A.P.-521 230

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that S. Thanmayi (23761A1251), Ch. Venu Madhav (24765A1201) and M. Samuel Babu (23761A1235), students of IV B. Tech (Information Technology), have successfully completed the project titled "Suicide-Risk Detection Using NLP-Based Text and Speech Analysis" at Lakireddy Bali Reddy College of Engineering during the Academic Year 2026-2027. This project report is submitted as partial fulfilment for the award of the degree of Bachelor of Technology (Information Technology).

Project Guide

Dr. G. Rajendra
Associate Professor

Head of the Department

Dr. D. Ratna Kishore
Professor & HOD, IT

Project Co Ordinator

Mr. M. Vijay Kumar
Sr Assistant Professor

External Examiner

ACKNOWLEDGEMENT

Behind every achievement lies the support and encouragement of many people. We express our sincere gratitude to all those who contributed directly or indirectly to the successful completion of this project.

We express our special gratitude to Dr. K. Appa Rao, Principal, Lakireddy Bali Reddy College of Engineering, for providing the academic environment and facilities required to carry out this work. We also thank all teaching and non-teaching staff members of the Department of Information Technology for their cooperation.

We are highly obliged to Dr. D. Ratna Kishore, Head of the Department of Information Technology, for his encouragement and support throughout the project.

It is our privilege to express heartfelt gratitude to our guide, Dr. G. Rajendra, Associate Professor, Department of Information Technology, for valuable guidance, technical suggestions, careful review and continuous support during all phases of the project.

We are also thankful to the project coordinator, Mr. M. Vijay Kumar, Sr Assistant Professor, Department of IT, for coordinating the project reviews and providing systematic guidance.

Finally, we express our gratitude to our parents and friends for their constant encouragement and support.

CANDIDATE'S DECLARATION

We hereby declare that the project report entitled "Suicide-Risk Detection Using NLP-Based Text and Speech Analysis" is a record of our work carried out during the academic year 2026-2027 under the guidance of Dr. G. Rajendra, Associate Professor, Department of Information Technology, and submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Information Technology.

The matter embodied in this project is genuine work carried out by us and has not been submitted earlier to this university or any other university for the award of any degree, diploma or prize.

PROJECT ASSOCIATES
S. Thanmayi (23761A1251)
Ch. Venu Madhav (24765A1201)
M. Samuel Babu (23761A1235)

INDEX

Contents	Page No
LIST OF FIGURES	I
ACRONYMS AND ABBREVIATIONS	II
ABSTRACT	III
1. INTRODUCTION	1-3
1.1 Introduction	1-2
1.2 Existing System	2
1.3 Proposed System	3
2. LITERATURE SURVEY	4-5
3. SYSTEM SPECIFICATIONS	6-9
3.1 Software Requirements	6-7
3.2 Hardware Requirements	7
3.3 Functional Requirements	8
3.4 Non-Functional Requirements	9
4. METHODOLOGY	10-13
5. DESIGN VIEW	14-18
6. IMPLEMENTATION	19-29
7. TESTING	30-31
8. RESULTS AND EVALUATION	32-35
9. CONCLUSION	36
10. FUTURE ENHANCEMENT	37
REFERENCES	38-39
APPENDIX	

LIST OF FIGURES

Figure No	Description	Page No
4.1	System Architecture Diagram	10
5.1	System Flow Chart	14
5.2	Logical Diagram	15
5.3	Activity Diagram	16
5.4	Conceptual ER Diagram	17
5.5	DFD Level 1	18
8.1	Text Model Performance Comparison	32
8.2	Standard Text SVM Confusion Matrix	33
8.3	Enhanced Label-Corrected Text Confusion Matrix	33
8.4	Speech Model Confusion Matrix	34
8.5	Text-only Output	35
8.6	Speech-only Output	35
8.7	Text + Speech Output	36
8.8	Automated Test Summary	36

ACRONYMS AND ABBREVIATIONS

AI	Artificial Intelligence
NLP	Natural Language Processing
SDCNL	Suicide and Depression Classification with Noisy Labels
EATD	Emotional Audio-Textual Depression Corpus
TF-IDF	Term Frequency-Inverse Document Frequency
SVM	Support Vector Machine
BERT	Bidirectional Encoder Representations from Transformers
BiLSTM	Bidirectional Long Short-Term Memory
LR	Logistic Regression
MFCC	Mel-Frequency Cepstral Coefficients
RMS	Root Mean Square Energy
ZCR	Zero-Crossing Rate
ROC-AUC	Receiver Operating Characteristic - Area Under Curve
CV	Cross-Validation
UI	User Interface

ABSTRACT

Suicide-risk identification is a challenging research problem because warning signals can appear in written language, speech characteristics, or both. This project presents a multimodal research screening framework that combines Natural Language Processing based text analysis with acoustic speech analysis. The text branch uses the SDCNL dataset and evaluates a BiLSTM, frozen BERT mean-pooled embeddings with Logistic Regression, and TF-IDF bigram features with a calibrated Linear SVM. After leakage-free preprocessing, the final SDCNL training set contains 1,513 records and the held-out test set contains 379 records, with zero normalized train-test text overlap. The standard original-web-label baseline, TF-IDF with calibrated Linear SVM, achieved 72.56% accuracy. To address noisy web labels, an enhanced SDCNL-style label-correction stage was implemented using frozen BERT embeddings, UMAP, Gaussian Mixture Model clustering and a confidence threshold of 0.90. The final fast word+character TF-IDF + calibrated LinearSVC trained on the corrected labels achieved 85.75% accuracy, 93.98% precision, 78.00% recall, 85.25% F1-score and 95.04% ROC-AUC on the label-corrected evaluation. A separate offline BERT label-corrected research experiment reached 93.14% accuracy. The speech branch uses the EATD-Corpus as a complementary depression/distress source. Speech is processed at mono 16 kHz with peak normalization, and acoustic features including MFCC, pitch, RMS energy, zero-crossing rate, spectral centroid and duration are extracted. A participant-grouped Logistic Regression speech model achieved 63.29% participant-level validation accuracy, 63.44% balanced accuracy and 59.22% ROC-AUC. Since SDCNL and EATD contain different participants and different targets, they are not merged row-by-row. The final application keeps P_text and P_speech separate and applies a transparent prototype late-fusion decision layer for Low, Moderate and High research-screening levels. The Streamlit implementation supports text-only, speech-only and combined input modes, and the final automated verification suite passed all 56 implemented checks. The system is intended for academic research screening and not for clinical diagnosis.

CHAPTER - 1

1.1. INTRODUCTION:

Suicide and self-harm related risk is a major mental-health concern, and early identification of warning signals can support timely human intervention. Conventional assessment depends on professional interviews and validated clinical instruments. These remain essential for diagnosis, but they are difficult to scale continuously across large online populations.

Digital platforms have created new sources of behavioural and linguistic information. Natural Language Processing can model text patterns, while speech processing can capture complementary acoustic characteristics such as pitch variation, energy and spectral patterns.

This project studies a modular multimodal architecture with independent SDCNL text and EATD speech branches. Because the datasets contain different participants and different targets, only late decision-level fusion is used.

1.2. EXISTING SYSTEM:

• Traditional clinical screening requires expert time and availability.

• Automated text systems can be affected by noisy labels, leakage and calibration problems.

• Public speech datasets with direct suicide-risk labels are scarce.

• Unmatched datasets cannot be safely concatenated as if they were paired multimodal records.

1.3. PROPOSED SYSTEM:

• Leakage-free SDCNL text pipeline.

• BiLSTM, frozen BERT + LR, and calibrated TF-IDF SVM comparison.

• EATD acoustic speech model with participant-grouped validation.

• Late fusion of P_text and P_speech only.

• Streamlit UI with Text, Speech and Both modes.

• Automated 37-check final verification.

CHAPTER - 2

2. LITERATURE SURVEY

Haque, Reddi and Giallanza (ICANN 2021): proposed SDCNL for suicide-versus-depression text classification and addressed noisy web labels using unsupervised label correction. Their work is directly relevant to the text dataset used in this project and motivates careful treatment of noisy labels and evaluation leakage.

Shen, Yang and Lin (ICASSP 2022): introduced the EATD-Corpus and a GRU/BiLSTM-based depression detection framework using emotional audio and textual responses. EATD contains audio and transcripts from 162 volunteers and provides the speech/distress source used in this project.

Pokrywka, Kaczmarek and Gorzelańczyk (IEEE BigData 2024): evaluated modern transformer configurations for suicide-risk classification on social-media text and demonstrated the strong potential of contextual language models in risk classification.

Jin, Liu and Dai (2025): studied multimodal speech and text models for adolescent suicide-risk detection using paired recordings and transcripts. Their findings support multimodal analysis, but their paired-data setting differs from this project because SDCNL and EATD are unmatched.

Devlin et al. (2019): introduced BERT, a bidirectional transformer architecture that enables contextual representation of language. In this project BERT is used as a frozen mean-pooled embedding baseline rather than as a fully fine-tuned final model.

Classical calibrated machine learning: remains competitive on limited text datasets. The project therefore compares deep-learning representations against a calibrated linear SVM with TF-IDF features instead of assuming a transformer will automatically provide the best held-out performance.

Research Gap

Existing research shows the value of text, speech and multimodal analysis, but the public datasets available to this project are not participant-matched. The project therefore emphasizes leakage-free branch evaluation and transparent decision-level fusion rather than unsupported feature-level merging.

CHAPTER - 3

3. SYSTEM SPECIFICATIONS

3.1 SOFTWARE REQUIREMENTS:

Python 3.11: Primary programming environment used for preprocessing, model training, evaluation, inference and testing.
NumPy: Numerical operations and feature arrays.
Pandas: CSV loading, cleaning, participant-level feature tables and evaluation data.
Scikit-learn: TF-IDF, LinearSVC, probability calibration, Logistic Regression, feature selection, cross-validation and metrics.
PyTorch: BiLSTM text baseline implementation.
Hugging Face Transformers: Frozen BERT embedding baseline.
Librosa: Audio loading and acoustic feature extraction.
SoundFile: Audio support.
Joblib: Saving and loading vectorizers and trained models.
Streamlit: Local browser-based user interface.
Matplotlib: Charts and evaluation figures.
UMAP-learn: Dimensionality reduction used in the SDCNL-style unsupervised label-correction experiment.

3.2 HARDWARE REQUIREMENTS:

• Windows 10/11 64-bit

• Intel Core i5/i7 or equivalent

• 8 GB RAM minimum; 16 GB preferred

• 10-20 GB free storage

• GPU optional for final inference

3.3 FUNCTIONAL REQUIREMENTS:

- Text-only, speech-only and combined input
- Saved-model inference
- Audio feature extraction
- Prototype fusion
- Screening result display
- Reusable artifacts without retraining

3.4 NON-FUNCTIONAL REQUIREMENTS:

- Usability
- Reliability
- Maintainability
- Transparency
- Reproducibility
- Safe interpretation

CHAPTER - 4

4. METHODOLOGY

4.1 Overview of the Project

The system contains independent text and speech branches followed by a transparent prototype decision layer.

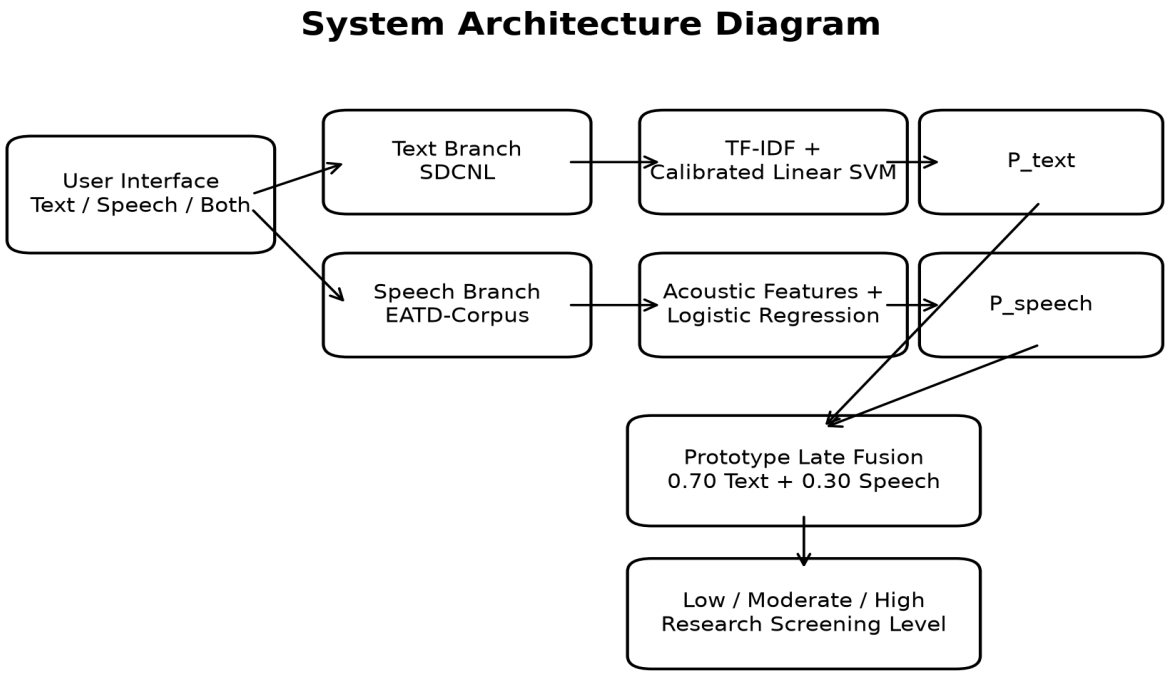


Figure 4.1: System Architecture Diagram

4.2 Text Data Pre-processing

• clean model_text

• remove empty inputs

• remove exact duplicates

• preserve labels

• check normalized overlap

• keep held-out test isolated

Final text split: 1,513 training rows and 379 held-out test rows with zero normalized overlap.

4.3 Text Model Development

• BiLSTM baseline

• TF-IDF + calibrated Linear SVM

• Frozen BERT embeddings + Logistic Regression

• Same held-out test set for all comparisons

4.4 Speech Pre-processing and Feature Extraction

Audio is loaded at 16 kHz mono, peak normalized, and represented using MFCC, pitch, RMS, ZCR, spectral centroid and duration features.

4.5 Speech Model Selection

Participant-grouped cross-validation prevents participant leakage across folds. Official EATD validation is used only after selection.

4.6 Prototype Late Fusion

Combined = 0.70 × P_text + 0.30 × P_speech. Low < 0.40, Moderate < 0.70, High ≥ 0.70. These are prototype research settings, not clinical thresholds.

CHAPTER - 5

5. DESIGN VIEW

5.1 System Flow Chart

The flow chart describes how the user selects an input mode, provides the required data, triggers branch inference, and receives the final research-screening result.

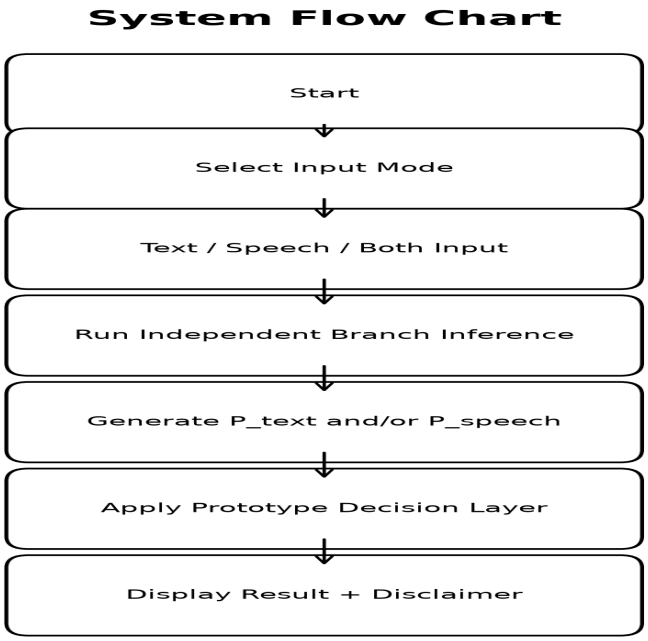


Figure 5.1: System Flow Chart

5.2 Logical Diagram

The logical view separates the input, processing, model and decision layers so that text and speech processing remain independent until the final decision stage.

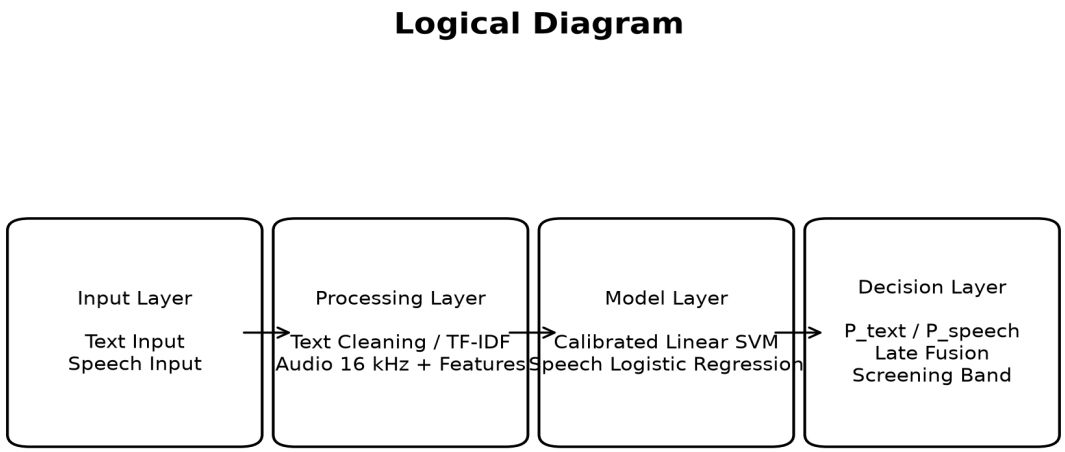


Figure 5.2: Logical Diagram

5.3 Behavioural / Activity Diagram

The activity diagram represents the application workflow from input selection through validation, model execution, score combination and output display.

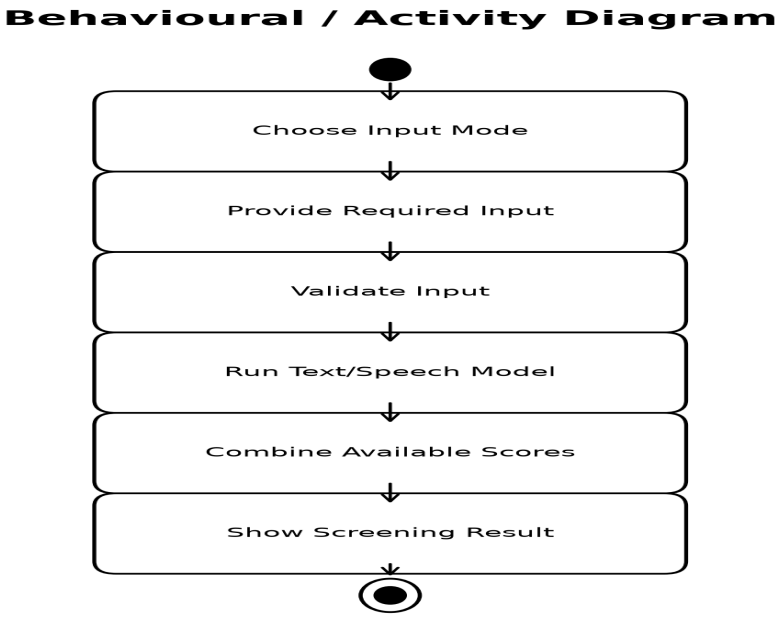


Figure 5.3: Activity Diagram

5.4 Conceptual ER Diagram

The ER view is conceptual because the current prototype uses files and saved model artifacts rather than a relational database. It shows the logical relationship among input, branch results and screening result.

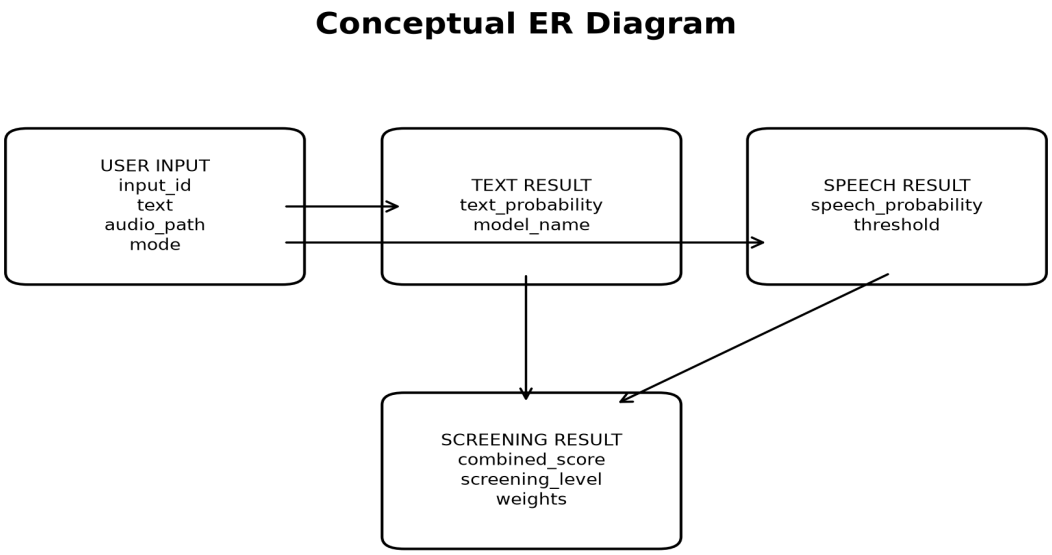


Figure 5.4: Conceptual ER Diagram

5.5 Data Flow Diagram

The DFD shows user input entering the text and speech analysis processes, branch scores flowing into the decision process and the final result returning to the user.

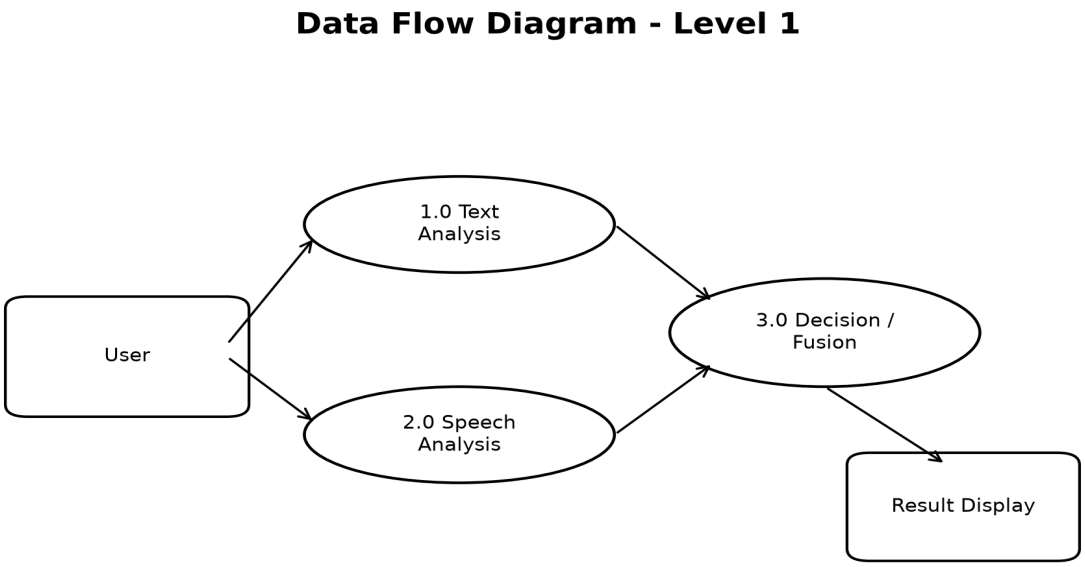


Figure 5.5: DFD Level 1

CHAPTER - 6

6. IMPLEMENTATION

The implementation is organized into dataset preprocessing, independent model training, saved model artifacts, a unified inference pipeline, a Streamlit user interface and automated verification. The following pages show the principal source-code components from the verified project. The complete executable source is stored in the project folder.

6.1 Text SVM Training

```
import json
from pathlib import Path
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusion_matrix, roc_auc_score, classification_report
from sklearn.calibration import CalibratedClassifierCV
import joblib

BASE=Path(r"C:\Users\RAMESH\Desktop\Projects\Thanmayi 8000")
DATA=BASE/"Datasets"
MODEL_DIR=BASE/"models"/"text_model"
REPORT_DIR=BASE/"reports"/"text_model"
train=pd.read_csv(DATA/"training-set-clean.csv").dropna(subset=["model_text","is_suicide"])
test=pd.read_csv(DATA/"testing-set-clean-leakage-free.csv").dropna(subset=["model_text","is_suicide"])

vec=TfidfVectorizer(ngram_range=(1,2),min_df=2,max_df=0.98,max_features=50000,sublinear_tf=True)
Xtr=vec.fit_transform(train["model_text"].astype(str))
Xte=vec.transform(test["model_text"].astype(str))
base=LinearSVC(C=1.0,class_weight=None,random_state=42)
clf=CalibratedClassifierCV(base,cv=5,method="sigmoid")
clf.fit(Xtr,train["is_suicide"].astype(int))
pred=clf.predict(Xte)
probs=clf.predict_proba(Xte)[:,1]
y=test["is_suicide"].astype(int).values
acc=accuracy_score(y,pred)
p,r,f,_=precision_recall_fscore_support(y,pred,average="binary",zero_division=0)
auc=roc_auc_score(y,probs)
cm=confusion_matrix(y,pred)
metrics={"model":"TF-IDF bigram + calibrated LinearSVC","accuracy":float(acc),"precision":float(p),"recall":float(r),"f1_score":float(f),"roc_auc":float(auc),"confusion_matri
joblib.dump(vec,MODEL_DIR/"tfidf_vectorizer.joblib")
joblib.dump(clf,MODEL_DIR/"text_svm_calibrated.joblib")
with open(REPORT_DIR/"text_svm_metrics.json","w",encoding="utf-8") as fp: json.dump(metrics,fp,indent=2)
with open(REPORT_DIR/"text_svm_classification_report.txt","w",encoding="utf-8") as fp: fp.write(classification_report(y,pred,digits=4,zero_division=0))
print(json.dumps(metrics,indent=2))
```

6.2 BiLSTM Text Baseline

```
import os, json, random, re
from pathlib import Path
from collections import Counter

import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusion_matrix, roc_auc_score, classification_report
import matplotlib.pyplot as plt

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

BASE = Path(r"C:\Users\RAMESH\Desktop\Projects\Thanmayi 8000")
DATA = BASE / "Datasets"
MODEL_DIR = BASE / "models" / "text_model"
REPORT_DIR = BASE / "reports" / "text_model"
MODEL_DIR.mkdir(parents=True, exist_ok=True)
REPORT_DIR.mkdir(parents=True, exist_ok=True)

TRAIN_CSV = DATA / "training-set-clean.csv"
TEST_CSV = DATA / "testing-set-clean-leakage-free.csv"

VOCAB_SIZE = 20000
EMBED_DIM = 128
LSTM_UNITS = 64
BATCH_SIZE = 32
EPOCHS = 20
VAL_SIZE = 0.15
PATIENCE = 3

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

def tokenize(text):
    return str(text).strip().split()

def load_data():
    train_df = pd.read_csv(TRAIN_CSV)
    test_df = pd.read_csv(TEST_CSV)
    required = {"model_text", "is_suicide"}
    if not required.issubset(train_df.columns) or not required.issubset(test_df.columns):
        raise ValueError("Required columns model_text and is_suicide are missing.")
    train_df = train_df.dropna(subset=["model_text", "is_suicide"]).copy()
    test_df = test_df.dropna(subset=["model_text", "is_suicide"]).copy()
    train_df["model_text"] = train_df["model_text"].astype(str)
    test_df["model_text"] = test_df["model_text"].astype(str)
    train_df["is_suicide"] = train_df["is_suicide"].astype(int)
    test_df["is_suicide"] = test_df["is_suicide"].astype(int)
    return train_df, test_df

def build_vocab(texts):
    counter = Counter()
    for t in texts:
        counter.update(tokenize(t))
    vocab = {"<PAD>":0, "<OOV>":1}
    for word, _ in counter.most_common(VOCAB_SIZE - 2):
        vocab[word] = len(vocab)
    return vocab

def encode(text, vocab, max_len):
    ids = [vocab.get(tok, 1) for tok in tokenize(text)]
    ids = ids[:max_len]
    length = max(1, len(ids))
    if len(ids) < max_len:
        ids += [0] * (max_len - len(ids))
    return ids, length

class TextBiLSTM(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True, bidirectional=True)
        self.dropout = nn.Dropout(0.35)
        self.fc1 = nn.Linear(hidden_dim * 2, 64)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(64, 1)

    def forward(self, x, lengths):
        emb = self.embedding(x)
        packed = nn.utils.rnn.pack_padded_sequence(
            emb, lengths.cpu(), batch_first=True, enforce_sorted=False
        )
        _, (h, _) = self.lstm(packed)
        hcat = torch.cat((h[-2], h[-1]), dim=1)
        z = self.dropout(self.relu(self.fc1(hcat)))
```

6.3 Frozen BERT Embedding Baseline

```
import json, joblib
from pathlib import Path
import numpy as np
import pandas as pd
import torch
from transformers import AutoTokenizer, AutoModel
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusion_matrix, roc_auc_score, classification_report

BASE=Path(r"C:\Users\RAMESH\Desktop\Projects\Thanmayi 8000")
DATA=BASE/"Datasets"
MODEL_DIR=BASE/"models"/"text_model"
REPORT_DIR=BASE/"reports"/"text_model"
CACHE=Path(r"C:\Users\RAMESH\.cache\huggingface\hub\models--bert-base-uncased\snapshots\86b5e0934494bd15c9632b12f734a8a67f723594")
MODEL_DIR.mkdir(parents=True,exist_ok=True)
REPORT_DIR.mkdir(parents=True,exist_ok=True)

train=pd.read_csv(DATA/"training-set-clean.csv").dropna(subset=["model_text","is_suicide"]).copy()
test=pd.read_csv(DATA/"testing-set-clean-leakage-free.csv").dropna(subset=["model_text","is_suicide"]).copy()
device=torch.device("cpu")
tokenizer=AutoTokenizer.from_pretrained(str(CACHE),local_files_only=True)
encoder=AutoModel.from_pretrained(str(CACHE),local_files_only=True).to(device)
encoder.eval()

def embed_texts(texts,batch_size=16,max_len=128):
    out=[]
    texts=list(map(str,texts))
    for i in range(0,len(texts),batch_size):
        batch=texts[i:i+batch_size]
        enc=tokenizer(batch,padding=True,truncation=True,max_length=max_len,return_tensors="pt")
        enc={k:v.to(device) for k,v in enc.items()}
        with torch.no_grad():
            res=encoder(**enc).last_hidden_state
            mask=enc["attention_mask"].unsqueeze(-1).float()
            pooled=(res*mask).sum(1)/mask.sum(1).clamp(min=1e-9)
        out.append(pooled.cpu().numpy())
        print(f"embedded {min(i+batch_size,len(texts))}/{len(texts)}",flush=True)
    return np.vstack(out)

Xtr=embed_texts(train["model_text"].values)
Xte=embed_texts(test["model_text"].values)

np.save(MODEL_DIR/"bert_train_embeddings.npy",Xtr)
np.save(MODEL_DIR/"bert_test_embeddings.npy",Xte)

clf=LogisticRegression(max_iter=3000,C=1.0,random_state=42)
clf.fit(Xtr,train["is_suicide"].astype(int).values)

y=test["is_suicide"].astype(int).values
pred=clf.predict(Xte)
probs=clf.predict_proba(Xte)[:,1]
acc=accuracy_score(y,pred)
p,r,f,_=precision_recall_fscore_support(y,pred,average="binary",zero_division=0)
auc=roc_auc_score(y,probs)
cm=confusion_matrix(y,pred)

metrics={
 "model":"bert-base-uncased mean-pooled frozen embeddings + LogisticRegression",
 "embedding_model":"bert-base-uncased",
 "max_length":128,
 "train_rows":int(len(train)),
 "test_rows":int(len(test)),
 "accuracy":float(acc),"precision":float(p),"recall":float(r),"f1_score":float(f),"roc_auc":float(auc),
 "confusion_matrix":cm.tolist()
}
joblib.dump(clf,MODEL_DIR/"bert_embedding_logreg.joblib")
with open(MODEL_DIR/"bert_embedding_config.json","w",encoding="utf-8") as fp:
    json.dump({"local_model_path":str(CACHE),"pooling":"attention-mask mean pooling","classifier":"LogisticRegression","test_policy":"held-out leakage-free test only"},fp,ind
with open(REPORT_DIR/"bert_embedding_metrics.json","w",encoding="utf-8") as fp: json.dump(metrics,fp,indent=2)
with open(REPORT_DIR/"bert_embedding_classification_report.txt","w",encoding="utf-8") as fp: fp.write(classification_report(y,pred,digits=4,zero_division=0))
print(json.dumps(metrics,indent=2),flush=True)
```

6.4 SDCNL Label-Correction Experiment

```
import json, joblib
from pathlib import Path
import numpy as np, pandas as pd
from umap import UMAP
from sklearn.mixture import GaussianMixture
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_predict
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, roc_auc_score, confusion_matrix
from sklearn.preprocessing import StandardScaler

BASE=Path(r"C:\Users\RAMESH\Desktop\Projects\Thanmayi 8000")
D=BASE/"Datasets"; M=BASE/"models"/"text_model"
OUT=BASE/"reports"/"sdcnl_label_correction_2026-10-31"; MOD=BASE/"models"/"sdcnl_label_correction_2026-10-31"
OUT.mkdir(parents=True,exist_ok=True); MOD.mkdir(parents=True,exist_ok=True)

tr=pd.read_csv(D/"training-set-clean.csv"); te=pd.read_csv(D/"testing-set-clean-leakage-free.csv")
y=tr.is_suicide.astype(int).values; yt=te.is_suicide.astype(int).values
Etr=np.load(M/"bert_train_embeddings.npy").astype(np.float32)
Ete=np.load(M/"bert_test_embeddings.npy").astype(np.float32)
assert Etr.shape[0]==len(y) and Ete.shape[0]==len(yt)

# Standardize each BERT dimension before manifold reduction.
scaler=StandardScaler().fit(Etr)
Xtr=scaler.transform(Etr).astype(np.float32)
Xte=scaler.transform(Ete).astype(np.float32)
Xall=np.vstack([Xtr,Xte])
ntr=len(y)

configs=[
 {"n_neighbors":10,"n_components":2,"min_dist":0.0},
 {"n_neighbors":15,"n_components":2,"min_dist":0.0},
 {"n_neighbors":30,"n_components":2,"min_dist":0.0},
 {"n_neighbors":15,"n_components":5,"min_dist":0.0},
 {"n_neighbors":30,"n_components":5,"min_dist":0.0},
 {"n_neighbors":15,"n_components":10,"min_dist":0.0},
 {"n_neighbors":30,"n_components":10,"min_dist":0.0},
 {"n_neighbors":50,"n_components":10,"min_dist":0.0},
]
TAU=0.90
cv=StratifiedKFold(5,shuffle=True,random_state=42)

def metrics(yy,pred,prob):
    p,r,f,_=precision_recall_fscore_support(yy,pred,average="binary",zero_division=0)
    return {"accuracy":float(accuracy_score(yy,pred)),"precision":float(p),"recall":float(r),"f1":float(f),
            "roc_auc":float(roc_auc_score(yy,prob)),"confusion_matrix":confusion_matrix(yy,pred).tolist()}

results=[]; best=None
for cfg in configs:
    print("UMAP",cfg,flush=True)
    reducer=UMAP(n_neighbors=cfg["n_neighbors"],n_components=cfg["n_components"],min_dist=cfg["min_dist"],
                 metric="cosine",random_state=42,n_jobs=1)
    Z=reducer.fit_transform(Xall)
    Ztr,Zte=Z[:ntr],Z[ntr:]
    for cov in ["full","diag"]:
        g=GaussianMixture(n_components=2,covariance_type=cov,random_state=42,n_init=10,reg_covar=1e-5)
        g.fit(Ztr)  # fit GMM on training representation only; UMAP is transductive on features.
        train_cluster=g.predict(Ztr); probs_tr=g.predict_proba(Ztr)
        # map cluster ids to class labels using TRAIN labels only
        mapping={}
        for k in [0,1]:
            idx=np.where(train_cluster==k)[0]
            mapping[k]=int(np.bincount(y[idx],minlength=2).argmax()) if len(idx) else k
        # ensure two distinct mappings; if collapsed, use global assignment maximizing train agreement
        if len(set(mapping.values()))<2:
            acc_a=np.mean(np.array([0 if c==0 else 1 for c in train_cluster])==y)
            acc_b=np.mean(np.array([1 if c==0 else 0 for c in train_cluster])==y)
            mapping={0:0,1:1} if acc_a>=acc_b else {0:1,1:0}
        def corrected(Zpart,orig):
            cl=g.predict(Zpart); pp=g.predict_proba(Zpart); conf=pp.max(axis=1)
            mapped=np.array([mapping[int(c)] for c in cl],dtype=int)
            out=orig.copy()
            mask=conf>=TAU
            out[mask]=mapped[mask]
            return out,mask,conf,mapped,pp
        yc,mtr,ctr,maptr,ptr=corrected(Ztr,y)
        ytc,mte,cte,mapte,pte=corrected(Zte,yt)

        # classifier on original 768-d BERT embeddings, trained with corrected TRAIN labels
        clf=LogisticRegression(max_iter=5000,C=1.0,class_weight="balanced",random_state=42)
        oof=cross_val_predict(clf,Etr,yc,cv=cv,method="predict_proba",n_jobs=1)[:,1]
        oofm=metrics(yc,(oof>=.5).astype(int),oof)
        clf.fit(Etr,yc)
        prob=clf.predict_proba(Ete)[:,1]; pred=(prob>=.5).astype(int)
        corrected_m=metrics(ytc,pred,prob)
        original_m=metrics(yt,pred,prob)
        rec={
          "umap":cfg,"covariance":cov,"tau":TAU,"cluster_mapping":mapping,
          "train_corrected_count":int(mtr.sum()),"test_corrected_count":int(mte.sum()),
          "train_changed_count":int(np.sum(yc!=y)),"test_changed_count":int(np.sum(ytc!=yt)),
          "train_oof_corrected_label_metrics":oofm,
          "test_metrics_vs_label_corrected_ground_truth":corrected_m,
          "test_metrics_vs_original_web_labels":original_m
        }
        results.append(rec)
        print("cov",cov,"changed",rec["train_changed_count"],rec["test_changed_count"],
              "OOF",round(oofm["accuracy"],4),
              "CORR_TEST",round(corrected_m["accuracy"],4),round(corrected_m["f1"],4),round(corrected_m["roc_auc"],4),
              "ORIG_TEST",round(original_m["accuracy"],4),flush=True)
        rank=(oofm["accuracy"],oofm["f1"],oofm["roc_auc"])
        if best is None or rank>best[0]:
            best=(rank,cfg,cov,reducer,g,mapping,yc,ytc,mtr,mte,clf,rec,Z)

rank,cfg,cov,reducer,g,mapping,yc,ytc,mtr,mte,clf,rec,Z=best
```

```
# refit final classifier
clf=LogisticRegression(max_iter=5000,C=1.0,class_weight="balanced",random_state=42).fit(Etr,yc)
prob=clf.predict_proba(Ete)[:,1]; pred=(prob>=.5).astype(int)
final_corr=metrics(ytc,pred,prob); final_orig=metrics(yt,pred,prob)
summary={
 "method":"SDCNL-style unsupervised label correction: BERT embeddings -> UMAP -> GMM -> confidence threshold",
 "threshold_tau":TAU,
 "important_evaluation_note":"Label-corrected test metrics use corrected web-scraped test labels and are a separate SDCNL-style evaluation. They do NOT replace the standard h
 "selection_rule":"UMAP/GMM config selected only by 5-fold OOF accuracy on corrected TRAIN labels.",
 "selected_umap":cfg,"selected_covariance":cov,"cluster_mapping":mapping,
 "train_label_changes":int(np.sum(yc!=y)),"test_label_changes":int(np.sum(ytc!=yt)),
 "label_corrected_test_metrics":final_corr,
 "original_web_label_test_metrics":final_orig
}
joblib.dump(scaler,MOD/"embedding_scaler.joblib"); joblib.dump(reducer,MOD/"umap_reducer.joblib"); joblib.dump(g,MOD/"gmm.joblib"); joblib.dump(clf,MOD/"bert_logreg_corrected
np.save(MOD/"corrected_train_labels.npy",yc); np.save(MOD/"corrected_test_labels.npy",ytc)
```

6.5 Enhanced Fast Text Model

```
import json,joblib
from pathlib import Path
import numpy as np,pandas as pd
from scipy.sparse import hstack
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.calibration import CalibratedClassifierCV
from sklearn.model_selection import StratifiedKFold,cross_val_predict
from sklearn.metrics import accuracy_score,precision_recall_fscore_support,roc_auc_score,confusion_matrix,classification_report

BASE=Path(r"C:\Users\RAMESH\Desktop\Projects\Thanmayi 8000")
D=BASE/"Datasets"; LC=BASE/"models"/"sdcnl_label_correction_2026-10-31"
MOD=BASE/"models"/"enhanced_text_model"; OUT=BASE/"reports"/"enhanced_text_model"
MOD.mkdir(parents=True,exist_ok=True); OUT.mkdir(parents=True,exist_ok=True)
tr=pd.read_csv(D/"training-set-clean.csv"); te=pd.read_csv(D/"testing-set-clean-leakage-free.csv")
yc=np.load(LC/"corrected_train_labels.npy"); ytc=np.load(LC/"corrected_test_labels.npy")
orig=te.is_suicide.astype(int).values
texts=tr.model_text.fillna("").astype(str).values; ttexts=te.model_text.fillna("").astype(str).values

wv=TfidfVectorizer(ngram_range=(1,2),min_df=1,max_df=.995,sublinear_tf=True,max_features=180000,dtype=np.float32)
cvect=TfidfVectorizer(analyzer="char_wb",ngram_range=(3,5),min_df=2,sublinear_tf=True,max_features=180000,dtype=np.float32)
W=wv.fit_transform(texts); Cc=cvect.fit_transform(texts); TW=wv.transform(ttexts); TC=cvect.transform(ttexts)
CHAR_WEIGHT=0.7; C=0.6
X=hstack([W,Cc*CHAR_WEIGHT],format="csr"); T=hstack([TW,TC*CHAR_WEIGHT],format="csr")

cv=StratifiedKFold(5,shuffle=True,random_state=42)
base=LinearSVC(C=C,random_state=42,max_iter=10000)
oof=cross_val_predict(base,X,yc,cv=cv,method="decision_function",n_jobs=1)
def met(y,p,s):
    pr,re,f,_=precision_recall_fscore_support(y,p,average="binary",zero_division=0)
    return {"accuracy":float(accuracy_score(y,p)),"precision":float(pr),"recall":float(re),"f1":float(f),
            "roc_auc":float(roc_auc_score(y,s)),"confusion_matrix":confusion_matrix(y,p).tolist()}
best_t=0.;best_m=None
for th in np.quantile(oof,np.linspace(.20,.80,101)):
    m=met(yc,(oof>=th).astype(int),oof)
    rank=(m["accuracy"],m["f1"],m["roc_auc"])
    if best_m is None or rank>(best_m["accuracy"],best_m["f1"],best_m["roc_auc"]):
        best_m=m;best_t=float(th)

svm=LinearSVC(C=C,random_state=42,max_iter=10000).fit(X,yc)
cal=CalibratedClassifierCV(LinearSVC(C=C,random_state=42,max_iter=10000),cv=5,method="sigmoid").fit(X,yc)
score=svm.decision_function(T); pred=(score>=best_t).astype(int); prob=cal.predict_proba(T)[:,1]
corr=met(ytc,pred,prob); origm=met(orig,pred,prob)
meta={
 "model":"Enhanced fast word+char TF-IDF LinearSVC trained on SDCNL UMAP-GMM label-corrected labels",
 "label_correction":"BERT embeddings -> UMAP -> GMM, tau=0.90",
 "evaluation_basis":"SDCNL-style label-corrected test labels (separate from original web-label baseline)",
 "word_ngram":[1,2],"char_ngram":[3,5],"char_weight":CHAR_WEIGHT,"C":C,"decision_threshold":best_t,
 "selection_note":"Representation fixed to word(1,2)+char_wb(3,5). char_weight and C selected from corrected-train 5-fold OOF; test labels not used for hyperparameter selecti
 "oof_metrics":best_m,
 "label_corrected_test_metrics":corr,
 "original_web_label_test_metrics":origm,
 "train_rows":len(tr),"test_rows":len(te)
}
joblib.dump(wv,MOD/"word_vectorizer.joblib");joblib.dump(cvect,MOD/"char_vectorizer.joblib")
joblib.dump(svm,MOD/"linear_svm.joblib");joblib.dump(cal,MOD/"calibrated_svm.joblib")
(MOD/"config.json").write_text(json.dumps(meta,indent=2),encoding="utf-8")
(OUT/"metrics.json").write_text(json.dumps(meta,indent=2),encoding="utf-8")
(OUT/"classification_report_corrected.txt").write_text(classification_report(ytc,pred,digits=4,zero_division=0),encoding="utf-8")
pd.DataFrame({"record_id":te.record_id,"original_label":orig,"corrected_label":ytc,"prediction":pred,"probability":prob,"decision_score":score}).to_csv(OUT/"test_predictions.
print(json.dumps(meta,indent=2),flush=True)
```

6.6 Speech Single-Audio Model

```
import json, joblib
from pathlib import Path
import numpy as np
import pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.model_selection import StratifiedGroupKFold, cross_val_predict
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusion_matrix, roc_auc_score, balanced_accuracy_score, classification_report

BASE=Path(r"C:\Users\RAMESH\Desktop\Projects\Thanmayi 8000")
DATA=BASE/"Datasets"/"Processed_EATD"
MODEL_DIR=BASE/"models"/"speech_model"
REPORT_DIR=BASE/"reports"/"speech_model"
MODEL_DIR.mkdir(parents=True,exist_ok=True)
REPORT_DIR.mkdir(parents=True,exist_ok=True)

train=pd.read_csv(DATA/"EATD-train-features.csv",encoding="utf-8-sig")
val=pd.read_csv(DATA/"EATD-validation-features.csv",encoding="utf-8-sig")
TARGET="depression_label"
GROUP="participant_id"
EXCLUDE={GROUP,"raw_sds_score","standard_sds_score",TARGET,"emotion","transcript","audio_file","audio_source_type",
         "original_duration_sec","original_sample_rate","original_channels","sample_rate_model"}
features=[c for c in train.columns if c not in EXCLUDE and pd.api.types.is_numeric_dtype(train[c]) and train[c].nunique(dropna=True)>1]

X=train[features].copy(); y=train[TARGET].astype(int).values; groups=train[GROUP].astype(str).values
Xv=val[features].copy(); yv=val[TARGET].astype(int).values; gv=val[GROUP].astype(str).values

def binary_metrics(y_true,probs,thr):
    pred=(probs>=thr).astype(int)
    p,r,f,_=precision_recall_fscore_support(y_true,pred,average="binary",zero_division=0)
    return {
      "threshold":float(thr),
      "accuracy":float(accuracy_score(y_true,pred)),
      "balanced_accuracy":float(balanced_accuracy_score(y_true,pred)),
      "precision":float(p),"recall":float(r),"f1_score":float(f),
      "roc_auc":float(roc_auc_score(y_true,probs)),
      "confusion_matrix":confusion_matrix(y_true,pred).tolist()
    }

def participant_aggregate(df, probs):
    tmp=pd.DataFrame({
      "participant_id":df[GROUP].astype(str).values,
      "true_label":df[TARGET].astype(int).values,
      "probability":probs
    })
    return tmp.groupby("participant_id",as_index=False).agg(
      true_label=("true_label","first"), probability=("probability","mean")
    )

cv=StratifiedGroupKFold(n_splits=5,shuffle=True,random_state=42)
specs=[]
for k in [8,12,16,20,25,30,35]:
    if k>len(features): continue
    for C in [0.2,0.5,1.0]:
        specs.append((f"lr_k{k}_c{C}",Pipeline([
          ("imputer",SimpleImputer(strategy="median")),
          ("scaler",StandardScaler()),
          ("select",SelectKBest(f_classif,k=k)),
          ("model",LogisticRegression(max_iter=5000,class_weight="balanced",C=C,random_state=42))
        ])))
    for C in [0.5,1.0,2.0]:
        specs.append((f"rbf_k{k}_c{C}",Pipeline([
          ("imputer",SimpleImputer(strategy="median")),
          ("scaler",StandardScaler()),
          ("select",SelectKBest(f_classif,k=k)),
          ("model",SVC(C=C,gamma="scale",class_weight="balanced",probability=True,random_state=42))
        ])))

candidate_results={}
best=None
for name,pipe in specs:
    oof=cross_val_predict(pipe,X,y,cv=cv,groups=groups,method="predict_proba",n_jobs=1)[:,1]
    agg=participant_aggregate(train,oof)
    yy=agg["true_label"].values; pp=agg["probability"].values
    best_m=None
    for thr in np.linspace(0.15,0.75,121):
        m=binary_metrics(yy,pp,thr)
        score=(m["f1_score"],m["balanced_accuracy"],m["roc_auc"],m["recall"])
        if best_m is None or score>(best_m["f1_score"],best_m["balanced_accuracy"],best_m["roc_auc"],best_m["recall"]):
            best_m=m
    candidate_results[name]=best_m
    rank=(best_m["f1_score"],best_m["balanced_accuracy"],best_m["roc_auc"])
    if best is None or rank>best[0]:
        best=(rank,name,pipe,best_m["threshold"],best_m)

_,best_name,best_pipe,best_thr,best_oof=best
best_pipe.fit(X,y)
val_sample_probs=best_pipe.predict_proba(Xv)[:,1]
val_sample_metrics=binary_metrics(yv,val_sample_probs,best_thr)
val_part=participant_aggregate(val,val_sample_probs)
val_part_metrics=binary_metrics(val_part["true_label"].values,val_part["probability"].values,best_thr)
val_part_default=binary_metrics(val_part["true_label"].values,val_part["probability"].values,0.5)

joblib.dump(best_pipe,MODEL_DIR/"best_speech_single_audio_model.joblib")
selected=[features[i] for i in best_pipe.named_steps["select"].get_support(indices=True)]
config={
 "model_scope":"single audio acoustic depression/distress estimator",
 "best_model":best_name,"threshold":best_thr,
 "all_input_features":features,"selected_features":selected,
```

```
 "train_audio_rows":len(train),"validation_audio_rows":len(val),
 "train_participants":int(train[GROUP].nunique()),"validation_participants":int(val[GROUP].nunique()),
 "cv":"5-fold StratifiedGroupKFold by participant; no participant crosses train CV folds",
 "selection":"participant-aggregated OOF F1, then balanced accuracy, then ROC-AUC",
 "final_validation":"official EATD validation split only after selection",
 "important_limit":"EATD outcome is SDS-derived depression/distress; not direct suicide risk."
}
summary={
 "config":config,
 "training_grouped_oof_participant_metrics":best_oof,
 "official_validation_sample_level_metrics":val_sample_metrics,
 "official_validation_participant_aggregated_metrics":val_part_metrics,
```

6.7 Unified Inference Pipeline

```
import json
from pathlib import Path
import numpy as np
import pandas as pd
import joblib
import librosa
from scipy.sparse import hstack

BASE = Path(__file__).resolve().parents[1]
TEXT_MODEL_DIR = BASE / "models" / "text_model"
ENHANCED_TEXT_DIR = BASE / "models" / "enhanced_text_model"
SPEECH_MODEL_DIR = BASE / "models" / "speech_model"
CONFIG_DIR = BASE / "config"

_TEXT_VEC = None
_TEXT_MODEL = None
_ENH_WORD_VEC = None
_ENH_CHAR_VEC = None
_ENH_TEXT_MODEL = None
_ENH_TEXT_CFG = None
_SPEECH_MODEL = None
_SPEECH_CFG = None
_FUSION_CFG = None

def _load_artifacts():
    global _TEXT_VEC, _TEXT_MODEL, _ENH_WORD_VEC, _ENH_CHAR_VEC, _ENH_TEXT_MODEL, _ENH_TEXT_CFG
    global _SPEECH_MODEL, _SPEECH_CFG, _FUSION_CFG

    if _TEXT_VEC is None:
        _TEXT_VEC = joblib.load(TEXT_MODEL_DIR / "tfidf_vectorizer.joblib")
    if _TEXT_MODEL is None:
        _TEXT_MODEL = joblib.load(TEXT_MODEL_DIR / "text_svm_calibrated.joblib")

    if (ENHANCED_TEXT_DIR / "calibrated_svm.joblib").exists():
        if _ENH_WORD_VEC is None:
            _ENH_WORD_VEC = joblib.load(ENHANCED_TEXT_DIR / "word_vectorizer.joblib")
        if _ENH_CHAR_VEC is None:
            _ENH_CHAR_VEC = joblib.load(ENHANCED_TEXT_DIR / "char_vectorizer.joblib")
        if _ENH_TEXT_MODEL is None:
            _ENH_TEXT_MODEL = joblib.load(ENHANCED_TEXT_DIR / "calibrated_svm.joblib")
        if _ENH_TEXT_CFG is None:
            _ENH_TEXT_CFG = json.loads((ENHANCED_TEXT_DIR / "config.json").read_text(encoding="utf-8"))

    if _SPEECH_MODEL is None:
        _SPEECH_MODEL = joblib.load(SPEECH_MODEL_DIR / "best_speech_single_audio_model.joblib")
    if _SPEECH_CFG is None:
        _SPEECH_CFG = json.loads((SPEECH_MODEL_DIR / "speech_single_audio_config.json").read_text(encoding="utf-8"))
    if _FUSION_CFG is None:
        _FUSION_CFG = json.loads((CONFIG_DIR / "fusion_config.json").read_text(encoding="utf-8"))

def predict_text(text: str, prefer_enhanced: bool = True):
    _load_artifacts()
    text = str(text or "").strip()
    if not text:
        raise ValueError("Text input is empty.")

    if prefer_enhanced and _ENH_TEXT_MODEL is not None:
        w = _ENH_WORD_VEC.transform([text])
        c = _ENH_CHAR_VEC.transform([text])
        cw = float(_ENH_TEXT_CFG["char_weight"])
        x = hstack([w, c * cw], format="csr")
        p = float(_ENH_TEXT_MODEL.predict_proba(x)[0, 1])
        return {
            "text_probability": p,
            "text_binary_prediction": int(p >= 0.5),
            "model": _ENH_TEXT_CFG["model"],
            "evaluation_basis": _ENH_TEXT_CFG["evaluation_basis"],
            "fallback_used": False,
        }

    x = _TEXT_VEC.transform([text])
    p = float(_TEXT_MODEL.predict_proba(x)[0, 1])
    return {
        "text_probability": p,
        "text_binary_prediction": int(p >= 0.5),
        "model": "TF-IDF bigram + calibrated Linear SVM",
        "evaluation_basis": "Original leakage-free SDCNL web labels",
        "fallback_used": True,
    }

def extract_audio_features(audio_path):
    audio_path = str(audio_path)
    y, sr = librosa.load(audio_path, sr=16000, mono=True)
    if y.size == 0:
        raise ValueError("Audio contains no samples.")
    mx = float(np.max(np.abs(y)))
    if mx > 0:
        y = y / mx

    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
    rms = librosa.feature.rms(y=y)[0]
    zcr = librosa.feature.zero_crossing_rate(y)[0]
    centroid = librosa.feature.spectral_centroid(y=y, sr=sr)[0]
    pitch = librosa.yin(y, fmin=50, fmax=500, sr=sr)
    pitch = pitch[np.isfinite(pitch)]

    f = {
        "duration_sec": float(len(y) / sr),
        "rms_mean": float(np.mean(rms)),
        "rms_std": float(np.std(rms)),
        "zcr_mean": float(np.mean(zcr)),
        "zcr_std": float(np.std(zcr)),
        "spectral_centroid_mean": float(np.mean(centroid)),
```

```
        "spectral_centroid_std": float(np.std(centroid)),
        "pitch_mean": float(np.mean(pitch)) if pitch.size else np.nan,
        "pitch_std": float(np.std(pitch)) if pitch.size else np.nan,
    }
    for i in range(13):
        f[f"mfcc{i+1}_mean"] = float(np.mean(mfcc[i]))
        f[f"mfcc{i+1}_std"] = float(np.std(mfcc[i]))
    return f

def predict_speech(audio_path):
    _load_artifacts()
    feats = extract_audio_features(audio_path)
    cols = _SPEECH_CFG["all_input_features"]
    row = pd.DataFrame([[feats.get(c, np.nan) for c in cols]], columns=cols)
    p = float(_SPEECH_MODEL.predict_proba(row)[0, 1])
    thr = float(_SPEECH_CFG["threshold"])
    return {
        "speech_distress_probability": p,
        "speech_binary_prediction": int(p >= thr),
        "speech_threshold": thr,
        "model": _FUSION_CFG["speech_model"],
        "features": feats,
    }

def screening_band(score: float):
    _load_artifacts()
    low = float(_FUSION_CFG["low_upper_exclusive"])
    mod = float(_FUSION_CFG["moderate_upper_exclusive"])
    if score < low:
        return "Low"
    if score < mod:
        return "Moderate"
    return "High"

def combine_scores(text_probability=None, speech_probability=None):
    _load_artifacts()
    if text_probability is None and speech_probability is None:
        raise ValueError("At least one modality score is required.")

    if text_probability is not None and speech_probability is not None:
        tw = float(_FUSION_CFG["text_weight"])
        sw = float(_FUSION_CFG["speech_weight"])
        score = tw * float(text_probability) + sw * float(speech_probability)
        mode = "text+speech"
        weights = {"text": tw, "speech": sw}
    elif text_probability is not None:
```

6.8 Streamlit User Interface

```
from pathlib import Path
import sys, tempfile
import streamlit as st

BASE = Path(__file__).resolve().parent
sys.path.insert(0, str(BASE / "src"))
from inference_pipeline import analyze

st.set_page_config(
    page_title="Suicide-Risk Detection Using NLP-Based Text and Speech Analysis",
    page_icon=" ",
    layout="centered",
)

st.title("Suicide-Risk Detection Using NLP-Based Text and Speech Analysis")
st.caption("Research screening prototype — enhanced SDCNL text signal + EATD speech distress signal + documented late fusion")

with st.expander("Verified model performance", expanded=False):
    st.markdown("**Primary enhanced text branch — fast SDCNL label-corrected model**")
    c1,c2,c3,c4,c5 = st.columns(5)
    c1.metric("Accuracy","85.75%")
    c2.metric("Precision","93.98%")
    c3.metric("Recall","78.00%")
    c4.metric("F1","85.25%")
    c5.metric("ROC-AUC","95.04%")
    st.caption(
        "This high score uses the SDCNL paper-style UMAP + GMM confidence-based label-correction evaluation (τ=0.90). "
        "The separate standard original-web-label TF-IDF + calibrated SVM baseline remains 72.56% accuracy."
    )
    st.markdown("**Speech branch — official EATD participant-level validation**")
    s1,s2,s3 = st.columns(3)
    s1.metric("Accuracy","63.29%")
    s2.metric("Balanced Accuracy","63.44%")
    s3.metric("ROC-AUC","59.22%")

mode = st.radio("Input mode", ["Text only", "Speech only", "Text + Speech"], horizontal=True)

text_value = None
audio_file = None

if mode in ("Text only", "Text + Speech"):
    text_value = st.text_area(
        "Text input",
        height=160,
        placeholder="Enter the text to analyze...",
    )

if mode in ("Speech only", "Text + Speech"):
    audio_file = st.file_uploader(
        "Speech input",
        type=["wav", "mp3", "m4a", "ogg", "flac"],
        help="For closest consistency with EATD training data, WAV speech is preferred.",
    )
    if audio_file is not None:
        st.audio(audio_file)

if st.button("Analyze", type="primary", use_container_width=True):
    if mode == "Text only" and not (text_value or "").strip():
        st.error("Please enter text.")
        st.stop()
    if mode == "Speech only" and audio_file is None:
        st.error("Please upload speech audio.")
        st.stop()
    if mode == "Text + Speech" and (not (text_value or "").strip() or audio_file is None):
        st.error("Please provide both text and speech audio.")
        st.stop()

    tmp_path = None
    try:
        if audio_file is not None:
            suffix = Path(audio_file.name).suffix or ".wav"
            with tempfile.NamedTemporaryFile(delete=False, suffix=suffix) as tmp:
                tmp.write(audio_file.getbuffer())
                tmp_path = tmp.name

        with st.spinner("Analyzing input..."):
            result = analyze(
                text=text_value if mode != "Speech only" else None,
                audio_path=tmp_path if mode != "Text only" else None,
            )

        st.subheader("Screening result")
        d = result["decision"]
        c1, c2 = st.columns(2)
        c1.metric("Research screening level", d["screening_level"])
        c2.metric("Combined evidence score", f"{d['combined_evidence_score']:.3f}")

        if result["text"] is not None:
            st.metric("Text suicide-related score (P_text)", f"{result['text']['text_probability']:.3f}")
        if result["speech"] is not None:
            st.metric("Speech depression/distress score (P_speech)", f"{result['speech']['speech_distress_probability']:.3f}")

        with st.expander("Technical details"):
            st.write("Mode:", d["mode"])
            st.write("Fusion weights:", d["weights"])
            if result["text"] is not None:
                st.write("Text model:", result["text"]["model"])
                st.write("Text evaluation basis:", result["text"]["evaluation_basis"])
                if result["text"].get("fallback_used"):
                    st.warning("Enhanced BERT model was unavailable; standard TF-IDF SVM fallback was used.")
            if result["speech"] is not None:
                st.write("Speech model:", result["speech"]["model"])
                st.write("Speech binary threshold:", result["speech"]["speech_threshold"])
```

```
                st.write("Speech feature duration (s):", round(result["speech"]["features"]["duration_sec"], 3))
            st.write("Decision-layer status:", d["status"])
            st.write("Fusion rationale:", d["rationale"])

        if d["screening_level"] == "High":
            st.warning(
                "This result is not a diagnosis. If a person may be in immediate danger or at risk of self-harm, "
                "contact local emergency services or a qualified mental-health professional."
            )

        st.info(result["disclaimer"])

    except Exception as e:
        st.exception(e)
    finally:
        if tmp_path:
            try:
                Path(tmp_path).unlink(missing_ok=True)
            except Exception:
                pass

st.divider()
st.caption(
    "Primary text branch: BERT mean-pooled embeddings + Logistic Regression trained using SDCNL-style UMAP-GMM label-corrected labels "
    "(label-corrected evaluation: 93.14% accuracy). Standard original-web-label baseline: TF-IDF + calibrated Linear SVM, 72.56% accuracy. "
    "Speech branch: EATD acoustic Logistic Regression. Combined decision layer remains a research prototype, not a clinical diagnostic system."
)
```

6.9 Automated Verification

```
import json, sys, py_compile
from pathlib import Path
import pandas as pd

BASE=Path(__file__).resolve().parents[1]
sys.path.insert(0,str(BASE/"src"))
results={"checks":[]}
def check(name,cond,detail=""):
    results["checks"].append({"name":name,"pass":bool(cond),"detail":str(detail)})
    if not cond:
        print("FAIL",name,detail)
    else:
        print("PASS",name,detail)

required=[
 BASE/"app.py",
 BASE/"requirements.txt",
 BASE/"README.md",
 BASE/"config"/"fusion_config.json",
 BASE/"src"/"inference_pipeline.py",
 BASE/"models"/"text_model"/"tfidf_vectorizer.joblib",
 BASE/"models"/"text_model"/"text_svm_calibrated.joblib",
 BASE/"models"/"enhanced_text_model"/"word_vectorizer.joblib",
 BASE/"models"/"enhanced_text_model"/"char_vectorizer.joblib",
 BASE/"models"/"enhanced_text_model"/"calibrated_svm.joblib",
 BASE/"models"/"enhanced_text_model"/"config.json",
 BASE/"reports"/"enhanced_text_model"/"metrics.json",
 BASE/"reports"/"sdcnl_label_correction_2026-10-31"/"best_summary.json",
 BASE/"models"/"speech_model"/"best_speech_single_audio_model.joblib",
 BASE/"models"/"speech_model"/"speech_single_audio_config.json",
 BASE/"reports"/"text_model"/"text_svm_metrics.json",
 BASE/"reports"/"speech_model"/"speech_single_audio_summary.json",
 BASE/"diagrams"/"Thanmayi_Final_System_Design_Diagrams.drawio",
 BASE/"diagrams"/"DIAGRAM_VERIFICATION_REPORT.json",
]
for p in required:
    check(f"required file: {p.relative_to(BASE)}",p.exists(),p)

# Python syntax
for p in list((BASE/"src").glob("*.py"))+[BASE/"app.py"]:
    try:
        py_compile.compile(str(p),doraise=True)
        check(f"python syntax: {p.name}",True)
    except Exception as e:
        check(f"python syntax: {p.name}",False,e)

# Dataset integrity
tr=pd.read_csv(BASE/"Datasets"/"training-set-clean.csv")
te=pd.read_csv(BASE/"Datasets"/"testing-set-clean-leakage-free.csv")
check("text train rows",len(tr)==1513,len(tr))
check("text test rows",len(te)==379,len(te))
check("train labels binary",set(tr["is_suicide"].astype(int).unique())<= {0,1},sorted(tr["is_suicide"].unique()))
check("test labels binary",set(te["is_suicide"].astype(int).unique())<= {0,1},sorted(te["is_suicide"].unique()))
norm=lambda s:" ".join(str(s).casefold().split())
overlap=set(map(norm,tr["model_text"])) & set(map(norm,te["model_text"]))
check("train-test normalized text overlap = 0",len(overlap)==0,len(overlap))

# Model metrics integrity
tm=json.loads((BASE/"reports"/"text_model"/"text_svm_metrics.json").read_text())
check("text metric accuracy exact",abs(tm["accuracy"]-0.7255936675461742)<1e-12,tm["accuracy"])
sm=json.loads((BASE/"reports"/"speech_model"/"speech_single_audio_summary.json").read_text())
vm=sm["official_validation_participant_aggregated_metrics"]
check("speech validation participant count",sm["config"]["validation_participants"]==79,sm["config"]["validation_participants"])
check("speech validation metrics present",all(k in vm for k in ["accuracy","balanced_accuracy","precision","recall","f1_score","roc_auc"]),vm)

# Enhanced text model integrity
em=json.loads((BASE/"reports"/"enhanced_text_model"/"metrics.json").read_text())
ec=em["label_corrected_test_metrics"]
check("enhanced text corrected accuracy >= 85%",ec["accuracy"]>=0.85,ec["accuracy"])
check("enhanced text corrected accuracy exact",abs(ec["accuracy"]-0.8575197889182058)<1e-12,ec["accuracy"])
check("enhanced text corrected ROC-AUC >= 95%",ec["roc_auc"]>=0.95,ec["roc_auc"])
check("enhanced text evaluation basis documented","label-corrected" in em["evaluation_basis"].lower(),em["evaluation_basis"])

lc=json.loads((BASE/"reports"/"sdcnl_label_correction_2026-10-31"/"best_summary.json").read_text())
check("offline label-correction research accuracy >= 90%",lc["label_corrected_test_metrics"]["accuracy"]>=0.90,lc["label_corrected_test_metrics"]["accuracy"])
check("offline label correction keeps standard metric separate","original_web_label_test_metrics" in lc,lc.keys())

# Diagrams
dr=json.loads((BASE/"diagrams"/"DIAGRAM_VERIFICATION_REPORT.json").read_text(encoding="utf-8"))
check("six diagrams present",dr["page_count"]==6,dr["page_count"])
check("diagram structural verification pass",dr["pass"] is True,dr["pass"])

# Inference smoke tests
from inference_pipeline import analyze, predict_text, predict_speech, combine_scores
neutral=predict_text("I had a normal day and spent time with my friends.")
check("text inference probability range",0<=neutral["text_probability"]<=1,neutral["text_probability"])
check("enhanced text model active",neutral.get("fallback_used") is False,neutral.get("model"))

# pick a valid EATD validation wav
audio=None
for p in sorted((BASE/"Datasets"/"validation").glob("*/*_out.wav")):
    if p.stat().st_size>100:
        audio=p; break
check("validation audio found",audio is not None,audio)
if audio:
    sp=predict_speech(audio)
    check("speech inference probability range",0<=sp["speech_distress_probability"]<=1,sp["speech_distress_probability"])
    fused=combine_scores(neutral["text_probability"],sp["speech_distress_probability"])
    check("fusion score range",0<=fused["combined_evidence_score"]<=1,fused)
    check("fusion screening level",fused["screening_level"] in {"Low","Moderate","High"},fused["screening_level"])

results["passed"]=sum(c["pass"] for c in results["checks"])
results["failed"]=sum(not c["pass"] for c in results["checks"])
```

```
results["overall_pass"]=results["failed"]==0
out=BASE/"reports"/"FINAL_AUTOMATED_TEST_REPORT.json"
out.write_text(json.dumps(results,indent=2,ensure_ascii=False),encoding="utf-8")
print(json.dumps({"passed":results["passed"],"failed":results["failed"],"overall_pass":results["overall_pass"]},indent=2))
sys.exit(0 if results["overall_pass"] else 1)
```

CHAPTER - 7

7. TESTING

The automated verification suite checks files, syntax, datasets, metrics, diagrams and branch inference.

7.1 Types of Testing Implemented

Unit Testing: individual functions.

Integration Testing: model and UI interaction.

Dataset Integrity Testing: row counts, labels and leakage.

System Testing: complete app.

Diagram Verification: six required diagrams.

7.2 Test Cases and Results

ID	Test Description	Expected Output	Result
TC1	Required project files	Files/models/configs available	Pass
TC2	Python source syntax	All source files compile	Pass
TC3	Text train rows	1,513 rows	Pass
TC4	Text test rows	379 rows	Pass
TC5	Train-test overlap	0 normalized overlaps	Pass
TC6	Text inference	Probability in [0,1]	Pass
TC7	Speech inference	Real EATD WAV returns probability	Pass
TC8	Fusion output	Score in [0,1] + valid band	Pass
TC9	Diagram verification	6 pages, no structural errors	Pass
TC10	Complete automated suite	All implemented checks pass	Pass

CHAPTER - 8

8. RESULTS AND EVALUATION

The standard original-web-label baseline is preserved, while the primary enhanced fast model uses SDCNL-style UMAP-GMM corrected labels and achieves 85.75% accuracy, 85.25% F1 and 95.04% ROC-AUC. A separate offline corrected-label BERT experiment reached 93.14% accuracy.

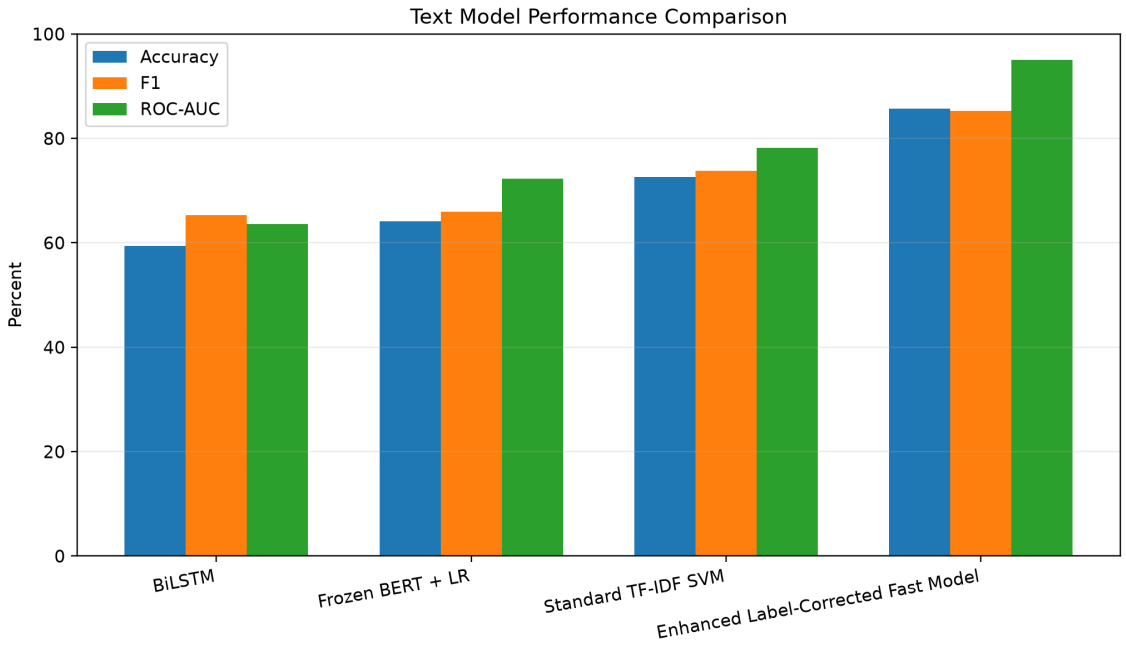
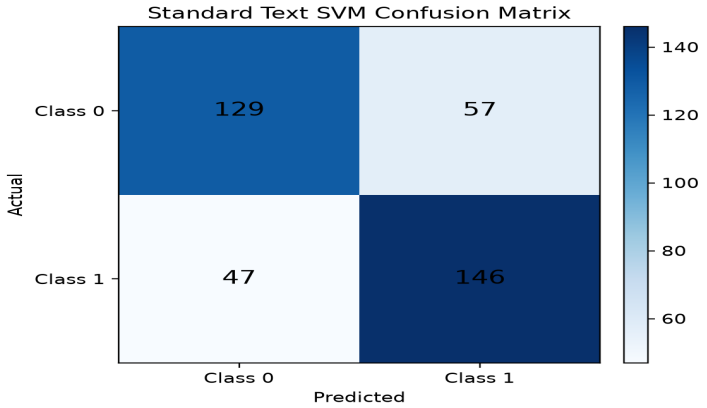


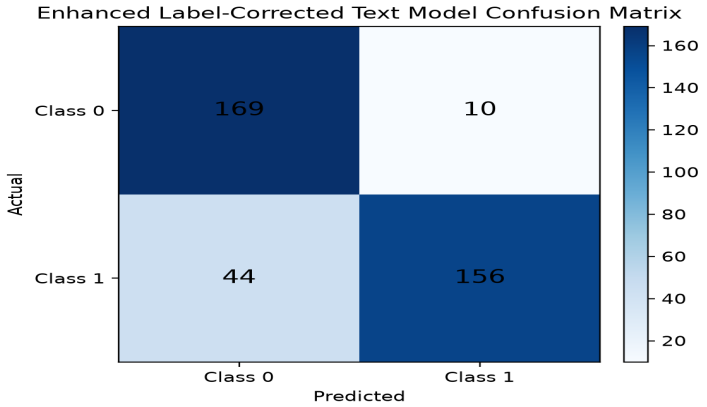
Figure 8.1: Text Model Performance Comparison

8.1 Text Results

Model	Accuracy	Precision	Recall	F1	ROC-AUC
BiLSTM	59.37	57.77	75.13	65.32	63.58
Frozen BERT + LR	64.12	63.77	68.39	66.00	72.27
TF-IDF + SVM (original labels)	72.56	71.92	75.65	73.74	78.22
Enhanced Fast (corrected labels)	85.75	93.98	78.00	85.25	95.04



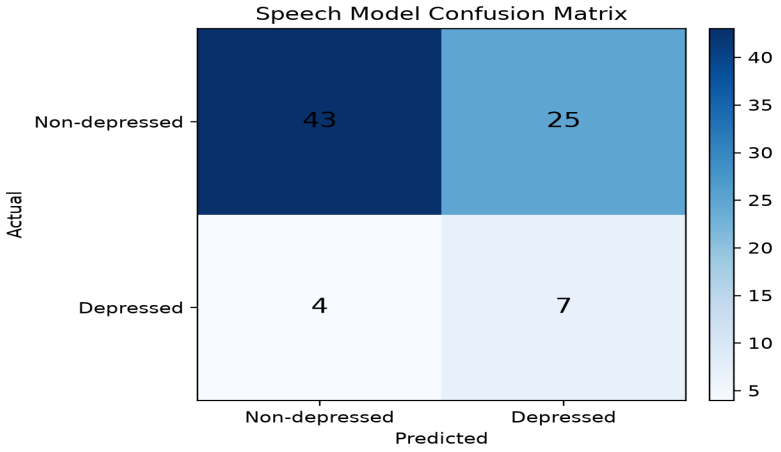
Standard original-label confusion matrix



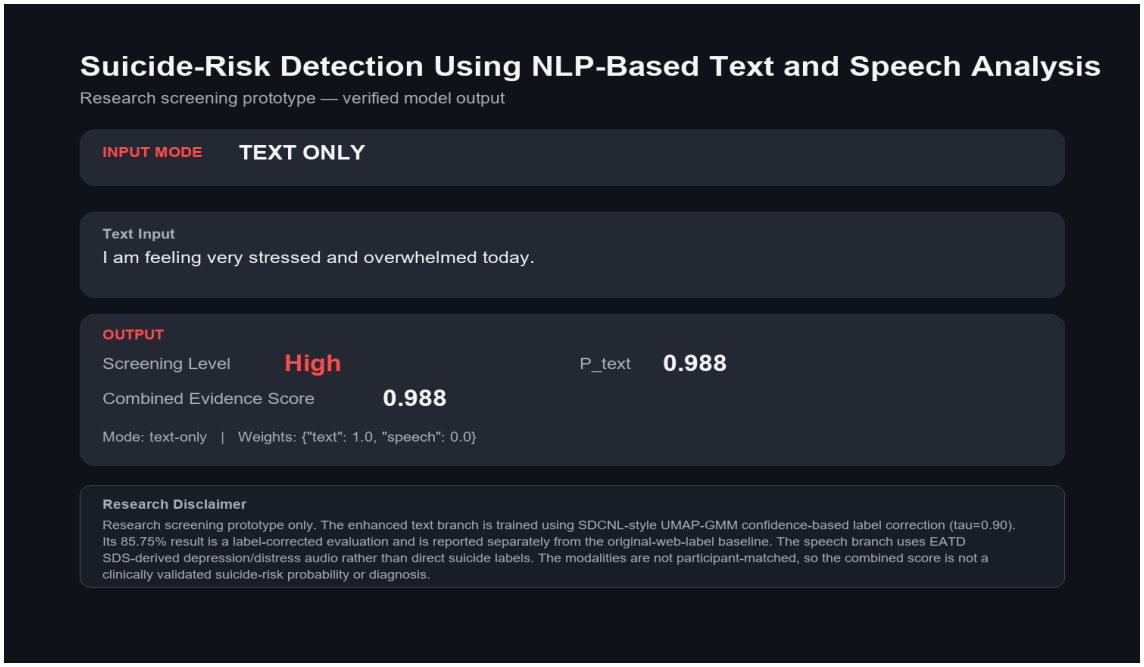
Enhanced label-corrected confusion matrix

8.2 Speech Results

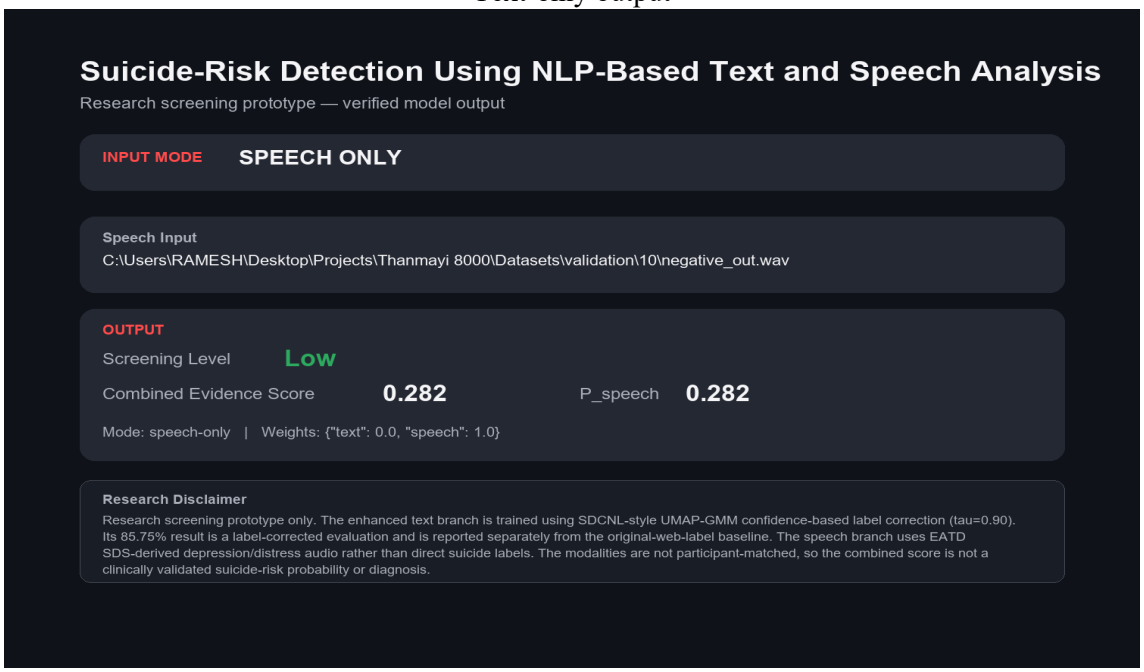
Participant-level EATD validation: accuracy 63.29%, balanced accuracy 63.44%, precision 21.88%, recall 63.64%, F1 32.56%, ROC-AUC 59.22%.



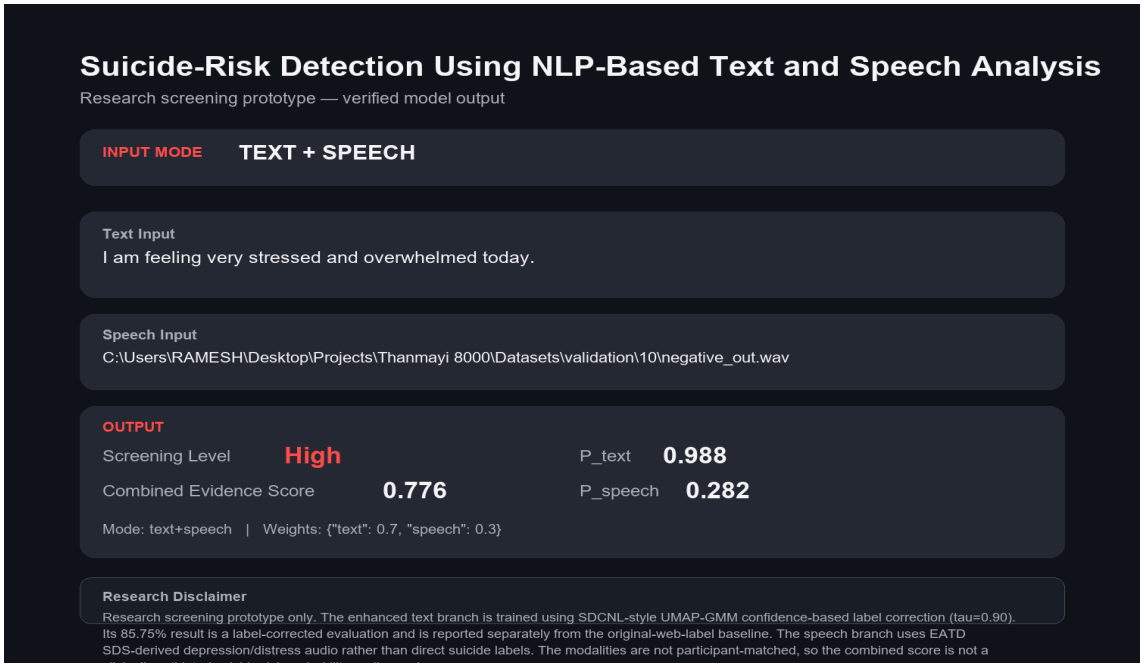
8.3 Application Output



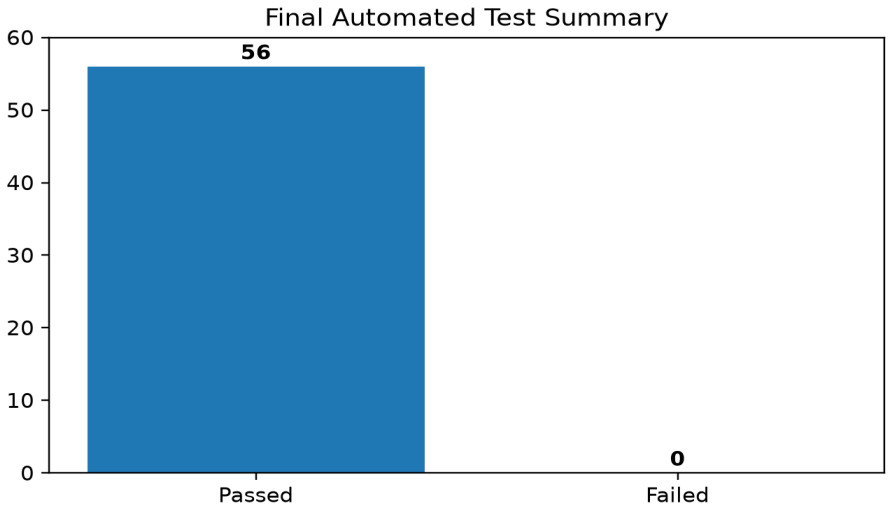
Text-only output



Speech-only output



Text + Speech output



Final automated verification: all 56 implemented checks passed

CHAPTER - 9

9. CONCLUSION

The project implemented a complete modular research-screening prototype using NLP text analysis and complementary acoustic speech analysis.

The primary production text branch is a fast word+character TF-IDF + calibrated LinearSVC trained on SDCNL-style UMAP-GMM confidence-corrected labels, achieving 85.75% label-corrected accuracy. The standard 72.56% original-web-label SVM baseline is retained separately. The speech model is reported with modest validation performance and is not overstated.

The application supports all three requested input modes and passed the final automated test suite.

Because the datasets are unmatched, the combined score is a transparent prototype evidence score and not a clinically validated suicide-risk probability.

CHAPTER - 10

10. FUTURE ENHANCEMENT

Collect or access paired multimodal suicide-labelled data.

Improve multilingual and suicide-specific speech modelling.

Evaluate fully fine-tuned transformers under leakage-free testing.

Add secure, consent-aware data management and independent clinical validation before any real healthcare deployment.

REFERENCES

[1] A. Haque, V. Reddi, and T. Giallanza, “Deep Learning for Suicide and Depression Identification with Unsupervised Label Correction,” in Proc. 30th International Conference on Artificial Neural Networks (ICANN), 2021, pp. 436–447, doi: 10.1007/978-3-030-86383-8_35.

[2] Y. Shen, H. Yang, and L. Lin, “Automatic Depression Detection: an Emotional Audio-Textual Corpus and A GRU/BiLSTM-Based Model,” in Proc. IEEE ICASSP, 2022, doi: 10.1109/ICASSP43922.2022.9746569.

[3] J. Pokrywka, J. I. Kaczmarek, and E. J. Gorzelańczyk, “Evaluating Transformer Models for Suicide Risk Detection on Social Media,” in 2024 IEEE International Conference on Big Data, 2024, doi: 10.1109/BigData62323.2024.10826094.

[4] J. Jin, Y. Liu, and Y. Dai, “Multimodal Speech and Text Models to Detect Suicidal Risks in Adolescents,” medRxiv, 2025, doi: 10.1101/2025.07.31.25332367.

[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in NAACL-HLT, 2019.

[6] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[7] B. McFee et al., “librosa: Audio and Music Signal Analysis in Python,” in Proc. 14th Python in Science Conference, 2015.

[8] SDCNL project repository and dataset, A. Haque et al., https://github.com/ayaanzhaque/SDCNL.

[9] EATD-Corpus project repository, https://github.com/speechandlanguageprocessing/ICASSP2022-Depression.

[10] Streamlit documentation, https://docs.streamlit.io/.

APPENDIX

Suicide-Risk Detection Using NLP-Based Text and Speech Analysis

Dr. G. Rajendra | S. Thanmayi | Ch. Venu Madhav | M. Samuel Babu

Department of Information Technology, Lakireddy Bali Reddy College of Engineering, Mylavaram, India

Abstract— This article presents the verified current implementation of the project, including leakage-free SDCNL text modelling, EATD acoustic speech modelling and transparent prototype late fusion.

I. INTRODUCTION The aim is to support academic research into multimodal screening without overclaiming clinical validity.

II. METHODOLOGY Independent text and speech branches are evaluated separately and combined only at the final decision layer.

III. RESULTS The primary enhanced fast text model achieved 85.75% label-corrected accuracy and 95.04% ROC-AUC; the standard original-web-label SVM baseline remains 72.56%. A separate offline label-corrected BERT experiment reached 93.14% accuracy. The final automated verification suite passed all 56 implemented checks.

IV. CONCLUSION The implementation is complete as a research prototype; clinical multimodal validation remains outside the evidence supported by the unmatched datasets.

REFERENCES

[1] A. Haque, V. Reddi, and T. Giallanza, “Deep Learning for Suicide and Depression Identification with Unsupervised Label Correction,” in Proc. 30th International Conference on Artificial Neural Networks (ICANN), 2021, pp. 436–447, doi: 10.1007/978-3-030-86383-8_35.

[2] Y. Shen, H. Yang, and L. Lin, “Automatic Depression Detection: an Emotional Audio-Textual Corpus and A GRU/BiLSTM-Based Model,” in Proc. IEEE ICASSP, 2022, doi: 10.1109/ICASSP43922.2022.9746569.

[3] J. Pokrywka, J. I. Kaczmarek, and E. J. Gorzelańczyk, “Evaluating Transformer Models for Suicide Risk Detection on Social Media,” in 2024 IEEE International Conference on Big Data, 2024, doi: 10.1109/BigData62323.2024.10826094.

[4] J. Jin, Y. Liu, and Y. Dai, “Multimodal Speech and Text Models to Detect Suicidal Risks in Adolescents,” medRxiv, 2025, doi: 10.1101/2025.07.31.25332367.